LAB REPORT VIVA 1: FLOW CONTROL

Team 5: Business Buzz

1. Name : Shaif Ahmad Bin Tufail Ahmad
   Matric no : 24067901
2. Name : Tanvir Hossain
   Matric no : 23121478
3. Name : Vosshindran A/L Thiyagaraja
   Matric no : 24004539
4. Name : Sifat Ul Islam Babor
   Matric no : 24064422
5. Name : Karl Xavier Franky
   Matric no : 23002044
6. Name : Osman Mohammedelamin Elsharief Ahmed
   Matric no : 23103329

Question - 1:

SECTION	DESCIRPTION
1. Problem	In the world of Ravnora, a username can indicate whether its owner is a friend or foe. This is determined by counting the distinct characters (runes) in the username: • If the number of distinct characters is even, the user is considered a friend. • If the number of distinct characters is odd, the user is considered a foe. Given a username string containing lowercase English letters, we need to output "ALLY DETECTED!" if the user is a friend, or "ENEMY ALERT!" if they are foes.
2. Solution	1. Use the indexOf method to check if a character has appeared earlier in the string. If the current character's first occurrence matches its current position in the string, it's a distinct character. 2. For each distinct character found, increment the counter numDistinctRunes. After the loop finishes, numDistinctRunes will hold the total number of unique characters in the username. 3. Determine Friend or Foe: 4. If the count of distinct characters is even, print "ALLY DETECTED!". 5. If the count is odd, print "ENEMY ALERT!".
3. Sample input and output	Input: Elvanar Output: **ALLY DETECTED!** Input: gorgoth Output: **ENEMY ALERT!** Input: arcanor Output: **ENEMY ALERT**!

4. Source code	Link: Viva1Q1.java
	
	import java.util.Scanner;
	
	public class Viva1Q1 {
	  public static void main(String[] args) {
	  // Initialize the Scanner to take input from the user
	  Scanner scan = new Scanner(System.in);
	
	  // Prompt the user for a username
	  System.out.println("Enter the username: ");
	  String username = scan.nextLine().trim();
	  // Read and trim any whitespace
	  // .trim() erases extra whitespace in front and behind code
	
	  // Initialize variable to count distinct runes
	  int numDistinctRunes = 0;
	
	  // Loop through each character in the username
	  // for loop starts with i = 0, meaning it begins at the first character of username
	  // i++ increments i by 1 on each iteration, moving to the next character in the string
	  for (int i = 0; i < username.length(); i++) {

```
            // character from username will be assigned to "rune" based on index
            /* In each iteration, we use username.charAt(i) to get the character at the current
               position i in thr username and store it in the variable "rune". For example,
               if the username is "elvanar", in the first iteration (i = 0), rune will be 'e',
               in the second iteration (i = 1), rune will be '1', and so on */
            /* In a word, i = 0, rune = e
                        i = 1, rune = l... */
            char rune = username.charAt(i);

            // Check if this character's first occurrence is at the current index (i)
            /* If username = "elvanar", then:
               i = 0, rune = 'e', and username.indexOf('e') = 0, numDistinctRunes = 1
               i = 1, rune = 'l' and username.indexOf('l') = 1, numDistinctRunes = 2
               i = 2, rune = 'v' and username.indexOf('v') = 2, numDistinctRunes = 3
               i = 3, rune = 'a' and username.indexOf('a') = 3, numDistinctRunes = 4
               i = 4, rune = 'n' and username.indexOf('n') = 4, numDistinctRunes = 5
               i = 5, rune = 'a' and username.indexOf('a') = 3s, numDistinctRunes = 5
               i = 6, rune = 'r' and username.indexOf('r') = 6, numDistinctRunes = 6
            */
            if (username.indexOf(rune) == i) {
                numDistinctRunes++;  // Increment count for distinct runes
            }
        }

        // Check if the count of distinct runes is even or odd
        if (numDistinctRunes % 2 == 0) { // If even, it's an ally
            System.out.println("ALLY DETECTED!");
        } else { // Else, it's an enemy
            System.out.println("ENEMY ALERT!");
        }

        // Close the Scanner
        scan.close();
    }
}
```

Question – 2:

Selection	Description	
1. Problem	In this problem, a program needs to be created to simulate the game "LuluLala." The game is designed to reinforce number skills and logical thinking. The program should prompt the user to input a range of numbers, and then it should print each number within this range following specific rules: • Print "Lulu" if a number is a multiple of 3. • Print "Lala" if a number is a multiple of 5. • Print "LuluLala" if a number is a multiple of both 3 and 5. • Print the number itself if none of the above conditions are met.	
2. Solution	To solve this problem, we will use conditional statements and a loop. The steps involved are as follows: Input: Prompt the user to enter the starting and ending values. Loop through Range: Use a for loop to iterate from the starting value to the ending value. Conditionals: • If the number is divisible by both 3 and 5, print "LuluLala." • If the number is divisible only by 3, print "Lulu." • If the number is divisible only by 5, print "Lala." • If none of the above conditions are true, print the number itself.	
3. Sample Input and Output	Input: Enter the starting value: 1 Enter the ending value: 20	Output: 1 2 Lulu 4 Lala Lulu 7 8 Lulu Lala 11 Lulu 13 14 LuluLala 16 17 Lulu 19 Lala

4. Source Code	Link: Viva1Q2SchoolGame.java

```java
import java.util.Scanner;
public class Viva1Q2SchoolGame {
    public static void main(String[] args){
    Scanner input = new Scanner(System.in);

    // input starting value from the user
    System.out.println("Enter the starting value: ");
    int startingValue = input.nextInt();

    // input ending value from the user
    System.out.println("Enter the output value: ");
    int endingValue = input.nextInt();

    for (int i = startingValue; i <= endingValue; i++){
        //check if divisible by both 3 and 5 to eliminate
        //other possibilities
        if (i % 3 == 0 && i % 5 == 0){
            System.out.println("Lululala");
        }
        else if (i % 3 == 0) {
            System.out.println("Lulu");
        }
        else if (i % 5 == 0) {
            System.out.println("Lala");
        }
        else {
            System.out.println(i);
        }
    }
    input.close();
  }
}
```

Question – 3:

Section	Description
1. Problem	A triangle is defined by three angles. To be valid, the angles must sum to 180°, and each angle must be greater than 0°. The task is to write a Java program that takes three angles as input, checks if they form a valid triangle, and then classifies the triangle as equilateral, isosceles, scalene, or right-angled.
2. Solution	1. Read the three angles a, b and c 2. Verify if the angles correspond to a triangle by using formula by verifying if a+b+c=180°. If a+b+c=180°, then the shape in question is indeed a valid triangle. Else, it is not a valid triangle. 3. If they correspond to a triangle, verify if the triangle is equilateral, isosceles or scalene first. • An equilateral triangle has all angles equal to 60°. Verify if a and b and c are equal to 60° • An isosceles triangle has two angles that are the same. Verify if only two angles are the same, if the verification for equilateral triangle fails. • A scalene triangle has three angles that are all different from each other. If the equilateral and isosceles triangle verifications fail, then the triangle is scalene. 4. Verify if the triangle has a 90° angle. If it does, it is also a right-angled triangle.

3. Sample Input & Output Code	**Input**	**Output**
	60 60 60	The triangle is valid. This is an equilateral triangle
	90 90 90	The triangle is not valid.
	90 45 45	The triangle is valid. It is an isosceles triangle. This is a right-angled triangle.
	78 69 33	The triangle is valid. It is a scalene triangle.

4. Source Code

Link: Viva1Q3.java

```
import java.util.Scanner;

public class Viva1Q3 {

    public static void main(String[] args) {
        // Initialize Scanner to read input
        Scanner scan = new Scanner(System.in);

        // Read the three angles
        System.out.println("Enter the angles of the triangle: ");
        int a = scan.nextInt();
        int b = scan.nextInt();
        int c = scan.nextInt();

        // Check if the sum of angles equals 180 and all angles are greater than 0
        if ((a + b + c) == 180 && a > 0 && b > 0 && c > 0) {
            System.out.println("The triangle is valid.");

            // Check if the triangle is right-angled
```

```
            if (a == 90 || b == 90 || c == 90) {
                System.out.println("This is a right-angled triangle.");
            }

            // Check if the triangle is equilateral
            if (a == 60 && b == 60 && c == 60) {
                System.out.println("It is an equilateral triangle.");
            }
            // Check if the triangle is isosceles (any two angles are equal)
            else if (a == b || a == c || b == c) {
                System.out.println("It is an isosceles triangle.");
            }
            // If neither equilateral nor isosceles, it must be scalene
            else {
                System.out.println("It is a scalene triangle.");
            }
            /* By using if, else if, and else- nested loop in sequence,
              we can stop checking once we find the correct triangle type,
              avoiding unnecessary checks.*/
        } else {
            // If the angles do not form a triangle
            System.out.println("The triangle is not valid.");
        }

        // Close the scanner
        scan.close();
    }
}
```

Question – 4:

Section	Description
1.Problem	Input validation is a simple yet very important part of programming. Write a program that asks the user to input a positive integer. If the user does not comply (by inputting text, floating values, or negative integers), inform the user that their input is invalid, and re-prompt the user to input a valid positive integer until the user complies. When a valid integer is inputted, declare if it is even or odd.
2.Solution	• **Set Up Scanner for Input**: Initialize a Scanner object to capture user input. • **Define Loop for Input Prompt**: Use a do-while loop to repeatedly ask the user for a positive integer until they provide valid input. • **Handle Input Parsing with Error Handling**: 1. Inside the loop, read the input as a String. 2. Use a try-catch block to attempt parsing the input to an integer. 3. If parsing fails (e.g., due to text or floating-point values), catch NumberFormatException and print an error message, prompting the user to enter a valid integer. • **Check for Positive Integer**: 1. Once parsing succeeds, check if the integer is positive. 2. If the integer is negative, display an error message and continue the loop to prompt for input again. • **Use break Statement to Exit Loop**: If the user inputs a valid positive integer, use break to exit the loop. • **Determine Even or Odd**: 1. After exiting the loop, check if the integer is even or odd using the modulo operator (%). 2. Print a message indicating whether the number is even or odd.
3.Sample Input & Output	Input: -2 → Output: Invalid Integer Input: 2 → Output: Positive Integer Input: -9.8 → Output: Invalid Integer
4.Solution	Link: Viva1Q4.java

```
import java.util.Scanner;

public class Viva1Q4 {
   public static void main(String[] args){
      Scanner scan = new Scanner(System.in);
      int number;

      /* do-while loop repeatedly execute the code inside until the user
         enters a valid positive integer. The do-while loop ensures the code
         runs at least once.
         do {
            // loop body
```

```
        } while (condition);

        Why not other loop?
        -A while loop first checks the condition before executing the loop body.
        This means if the condition is false at the start, the body of the loop
        will not execute at all.
        while (condition) {
         // loop body
        }
        -A for loop is generally used for situations where the number
        of iterations is known or easily determined.
        for (initialization; condition; update) {
        // loop body
        }
      */
      do {
         System.out.print("Enter a positive integer: ");
         String input = scan.nextLine();
         // nextLine() reads everything that the user types, including spaces

         /* The try block is used to handle exceptions (in this case,
            specifically the NumberFormatException) that may occur when
            attempting to parse user input into an integer.  the try block is
            used to enclose code that might throw an exception (an error) during
execution.
            The try block is followed by one or more catch blocks, which specify
            how to handle different types of exceptions that might occur.
            try {
              // Code that may cause an exception
            } catch (ExceptionType e) {
              // Handle the exception
            }

            1. The code inside the try block is executed first.
            2. If an exception is thrown during the execution of this code, the program
will
               immediately jump to the appropriate catch block that matches the type of
exception.
            3. If no exception occurs, the catch block is skipped, and the program
continues normally. */
         try {
            number = Integer.parseInt(input);
            /* This try block attempts to convert the user's input (stored as a String)
               to an int using Integer.parseInt(input). If the input is a valid integer,
               it will be stored in the number variable. If the user enters non-numeric
               characters or a decimal, this line will throw a NumberFormatException,
               causing the code to jump to the catch block below, where we handle the
error. */

            // Ensure number is a positive integer greater than zero
            if (number <= 0) {
```

```
                System.out.println("Invalid input. Please enter a positive integer.");
                continue;
            /* This if statement checks if the number is not a positive integer (either zero
or negative).
                If number <= 0 is true, we display an error message, telling the user to
enter a positive integer,
                and use continue; to skip the remaining code in the loop and start the next
iteration. */
            }
            break;
            /* If the input passes all checks (it's a valid positive integer), this break
statement exits
                the do-while loop, allowing the program to continue to the next part. */

            /* This catch block catches any NumberFormatException thrown by
Integer.parseInt(input).
                This exception occurs if the input is not a valid integer (e.g., if the user
enters letters or special characters).
                catch (ExceptionType variableName) {
                    // Handle the exception
                }
                catch (ExceptionType variableName) {
                    // Handle the exception
                } */
        } catch (NumberFormatException e) {
            System.out.println("Invalid input. Please enter a valid integer.");
        }
    } while (true);
     /* Creates an infinite loop to repeatedly prompt the user.
        Relies on break to exit the loop only when a valid positive integer is entered.
*/

    // Check if the number is even or odd
    if (number % 2 == 0) {
        System.out.println(number + " is even.");
    } else {
        System.out.println(number + " is odd.");
    }

    scan.close();
  }
}
```

Question 5:

Section	Description
1. Problem	Given positive integer n (1 ≤ n ≤ 10^18) that has no leading zero. The task is grouping same digit that repeats consecutively. The groups should be in order of appearance. The goal is to determine the total number of digit groups, the digit that forms the longest group and its length (If there is a tie for the longest group, choose the digit that appears first), and the sum of lengths of all digit groups. The program should output the total number of groups, the digit forming the longest group and its length, and the sum of all group lengths, in that order.
2. Solution	1. **Input Validation**: • Ensure the input is a positive long integer (≤ 10^18). • Check there are no leading zeros. 2. **Initialize Variables**: • Set a counter for the total number of groups. • Track the longest group's length and its digit. • Set a sum variable for all group lengths. 3. **Iterate through Digits**: • Compare each digit with the next one. • If they match, increase the current group length. • If they don't match, add the current group length to the total sum, update the group count, and check if this group is the longest. 4. **Update Longest Group**: • Each time a new group is encountered, update the longest group if necessary. 5. **Final Output**: • Output the total number of groups. • Output the digit of the longest group and its length. • Output the sum of all group lengths.
3. Sample input and output	Input 11122221111 Output 3 2 4 11 Input 1223334444 Output 4 4 4 10

4. Source code	Link: Viva1Q5.java

```
import java.util.Scanner;

public class Viva1Q5 {

   public static void main(String[] args) {
      Scanner scan= new Scanner(System.in);

      // Check if input is within the range of long
      /* The line if (scan.hasNextLong()) is part of the Scanner class's input-checking
functionality,
         and it works by checking whether the next input entered by the user can be
interpreted as a long integer. */
      /* This checks if the next input from the user is a long integer (i.e., a whole
number within the range -2^63 to 2^63-1).
         If it's not a valid long, the program will skip to the else block at the end. */
      if(scan.hasNextLong()){
      long num = scan.nextLong(); // If the input is valid, we read the long number from
the user and store it in the variable num.

         /* This checks if num is a positive integer and does not exceed 10^18 (the
maximum allowed value).
         If num is out of this range, the program will print "Too long" and exit. */
         if (num > 0 && num <= 1000000000000000000L) {
            String n = Long.toString(num); /* convert num to a String and store it in n.
This makes it easier to work with individual digits
                                    since we can access each character of the string
separately. */
            int totalGroups = 0;
            int sumLength = 0;
            int longestLength = 0;
            char longestDigit = n.charAt(0);

            int currentLength = 1; // Start with the first digit

            // This for loop starts from the second character (index 1) and iterates
through each character in n.
            for (int i = 1; i < n.length(); i++) {
               if (n.charAt(i) == n.charAt(i - 1)) {
                  /* if the current character (n.charAt(i)) is the same as the previous
character (n.charAt(i - 1)).
                     If they match, it means we're still in the same group of identical
digits.h */
                     currentLength++; //f the characters match, we increase currentLength
by 1, extending the current group.
               } else {
                  // If the characters do not match, it means the current group has ended
                  totalGroups++;
                  sumLength += currentLength;
```

```
                // Check if this is the longest group
                if (currentLength > longestLength) {
                    longestLength = currentLength;
                    longestDigit = n.charAt(i - 1);
                }

                // Reset current length for the new group
                currentLength = 1;
            }
        }

        // Add the last group
        totalGroups++;
        sumLength += currentLength;
        if (currentLength > longestLength) {
            longestLength = currentLength;
            longestDigit = n.charAt(n.length() - 1);
        }

        // Print the results
        System.out.println(totalGroups);
        System.out.println(longestDigit + " " + longestLength);
        System.out.println(sumLength);

    } else {
        System.out.println("Too long");
    }
} else {
    System.out.print("Too much");
}
}
}
```

Question – 6:

Section	Description
1. Problem	The goal of this program is to print the word "MALAYSIA" in a specific star (*) pattern format. Each letter of the word is represented in a 7x7 grid, where specific cells in each grid are filled with * characters to form the shape of each letter. The program should use loops and conditional statements to print each letter horizontally, with spaces between them to maintain alignment. Pattern Specifications: • Each letter in "MALAYSIA" is printed within a 7x7 grid. • The * character is used to form each letter, with appropriate spacing between the letters. • Loops and conditional statements are used to construct each letter's pattern. • The letters are printed horizontally with spaces between them to keep the word aligned.
2. Solution	To solve this problem, we will: • Use a loop to iterate through each row (0 to 6). • For each row, use nested loops and conditional statements to check if a specific position (defined by row and column) should contain a * or a space. • Print a * or space in each cell to form each letter. • Add spacing between letters by using System.out.print(" ");. • Use System.out.println(); at the end of each row to move to the next line, thus keeping the 7x7 grid format for each letter.
3. Sample output	* * ******* * ******* * * ******* ******* ******* * * ** * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * ******* * ******* * ******* * ******* * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * ******* * * * ******* ******** * *

4. Source Code	Link: Viva1Q6.java

```
public class Viva1Q6 {

    public static void main(String[] args) {

        /* for loop that initializes i to 0 and runs until i < 77. This loop iterates 77
times, representing 77 columns across one row. This number
        accommodates each 7-column-wide letter- the word "MALAYSIA" has 8
letters (7*8 = 56) and the spaces (3-column space, 3*7 = 21) between them. */
        //1st row
        for (int i = 0; i < 77; i++) {
            if ((i == 0 || i == 6 || i == 20 || i == 40 || i == 46) || (i >= 10 && i <= 16) ||
(i >= 30 && i <= 36) ||
                (i >= 50 && i <= 56) || (i >= 60 && i <= 66) || (i >= 70 && i <= 76))
                System.out.print("*");
            else
                System.out.print(" ");
        }
        System.out.println();

        // 2nd row
        for (int i = 0; i < 77; i++) {
            if (i == 0 || i == 1 || i == 5 || i == 6 || i == 10 || i == 16 || i == 20 || i == 30 ||
i == 36 ||
                i == 41 || i == 45 || i == 50 || i == 63 || i == 70 || i == 76)
                System.out.print("*");
            else
                System.out.print(" ");
        }
        System.out.println();

        // 3rd row
        for (int i = 0; i < 77; i++) {
            if (i == 0 || i == 2 || i == 4 || i == 6 || i == 10 || i == 16 || i == 20 || i == 30 ||
i == 36 ||
                i == 42 || i == 44 || i == 50 || i == 63 || i == 70 || i == 76)
                System.out.print("*");
            else
                System.out.print(" ");
        }
        System.out.println();

        // 4th row
        for (int i = 0; i < 77; i++) {
            if (i == 0 || i == 3 || i == 6 || (i >= 10 && i <= 16) || i == 20 || (i >= 30 &&
i <= 36) ||
                i == 43 || (i >= 50 && i <= 56) || i == 63 || (i >= 70 && i <= 76))
                System.out.print("*");
            else
                System.out.print(" ");
```

```
        }
        System.out.println();

        // 5th row
        for (int i = 0; i < 77; i++) {
            if (i == 0 || i == 6 || i == 10 || i == 16 || i == 20 || i == 30 || i == 36 || i == 43
|| i == 56 ||
                i == 63 || i == 70 || i == 76)
                System.out.print("*");
            else
                System.out.print(" ");
        }
        System.out.println();

        // 6th row
        for (int i = 0; i < 77; i++) {
            if (i == 0 || i == 6 || i == 10 || i == 16 || i == 20 || i == 30 || i == 36 || i == 43
|| i == 56 ||
                i == 63 || i == 70 || i == 76)
                System.out.print("*");
            else
                System.out.print(" ");
        }
        System.out.println();

        // 7th row
        for (int i = 0; i < 77; i++) {
            if (i == 0 || i == 6 || i == 10 || i == 16 || (i >= 20 && i <= 26) || i == 30 || i
== 36 || i == 43 ||
                (i >= 50 && i <= 56) || (i >= 60 && i <= 67) || i == 70 || i == 76)
                System.out.print("*");
            else
                System.out.print(" ");
        }
        System.out.println();
    }
}
```